

Техническая документация системы мониторинга Raspberry Pi с датчиком расстояния.

1. Введение

Система предназначена для удалённого мониторинга состояния одноплатного компьютера Raspberry Pi (далее – RPi) с возможностью получения данных о температуре процессора, загрузке CPU и памяти, использовании дискового пространства, а также измерении расстояния до объекта с помощью ультразвукового датчика HC-SR04, подключённого к плате Arduino. Клиентское приложение под управлением Windows позволяет в реальном времени отслеживать показатели и просматривать журнал критических событий.

2. Архитектура системы

Система состоит из трёх независимых компонентов:

- Arduino – измеряет расстояние с помощью ультразвукового датчика и отправляет значения по последовательному интерфейсу (UART) на Raspberry Pi.
- Сервер (Raspberry Pi) – собирает системные метрики, принимает данные с Arduino по USB, ведёт журналы, обрабатывает команды от клиентов через TCP-сокеты.
- Клиент (Windows) – подключается к серверу, отправляет команды (LIVE или LOGS), отображает потоковую информацию в реальном времени или выводит историю критических событий.

Схема взаимодействия:

[Датчик HC-SR04] → [Arduino] → (UART / USB) → [Сервер на RPi] ← (TCP port 5000) → [Клиент Windows]

RPi периодически собирает системные метрики (интервал 5 секунд) и одновременно читает данные с последовательного порта /dev/ttyUSB0. При превышении пороговых значений (температура >70 °C, загрузка CPU >90 %, расстояние <20 см) генерируется запись в журнале событий.

Клиентское приложение устанавливает TCP-соединение с сервером и в зависимости от выбранного режима:

- LIVE – получает JSON-строки с обновлением каждые 2 секунды;
- LOGS – получает последние 50 записей из журнала критических событий.

3. Описание компонентов

3.1. Arduino (скетч)

Назначение: измерение расстояния ультразвуковым датчиком HC-SR04.

Параметры:

- Выводы: TRIG_PIN = 9, ECHO_PIN = 8.
- Скорость передачи данных: 9600 бод.
- Частота измерений: каждые 200 мс.
- Формат вывода: строка DIST:<значение> с одним десятичным знаком, например DIST:15.3. При отсутствии эха (duration = 0) выводится DIST:0.0.

Алгоритм:

Генерируется импульс на TRIG длительностью 10 мкс, измеряется длительность отклика на ECHO. Расстояние (см) = длительность * 0.0343 / 2.

3.2. Сервер (Raspberry Pi, C++)

Назначение: сбор метрик, обработка данных с Arduino, обслуживание клиентских запросов, ведение логов.

Основные потоки:

1. serial_read_loop – чтение из последовательного порта, парсинг строк вида DIST:..., сохранение последнего значения.
2. collect_loop – сбор системных метрик (каждые 5 секунд), проверка порогов, запись в журналы, обновление глобальной структуры SystemData.
3. handle_client – обработка каждого подключившегося клиента в отдельном потоке.

Пороговые значения (жёстко заданы в коде):

- Температура CPU: TEMP_THRESHOLD = 70.0 °C
- Загрузка CPU: CPU_THRESHOLD = 90.0 %
- Расстояние: DIST_THRESHOLD = 20.0 см (если расстояние меньше порога – событие)

Метрики системы:

- Температура CPU – чтение из /sys/class/thermal/thermal_zone0/temp.
- Загрузка CPU – вычисляется по данным /proc/stat (два замера с интервалом 1 сек).
- Использование ОЗУ – анализ /proc/meminfo (используется формула MemTotal – MemFree – Buffers – Cached).
- Использование диска – через statvfs` для корневой файловой системы.
- Расстояние – из последнего значения, полученного от Arduino.

Журналы:

- Директория: ~/pi_monitor/
- system.log – полный журнал всех измерений (время, статус, все метрики).
- critical_events.log – события превышения порогов с деталями.

Сетевой интерфейс:

- Порт: 5000 (TCP, IPv4)
- Обработка команд от клиента:
 - LIVE – начинает периодическую отправку JSON-данных (интервал 2 секунды).
 - LOGS – возвращает последние 50 строк из critical_events.log.
 - Любая другая команда возвращает ошибку.

Формат JSON для режима LIVE:

json

```
{ "time": "HH:MM:SS", "temp_c": 45.2, "cpu_pct": 12.5, "mem_pct": 34.1, "disk_pct": 28.3, "dist_cm": 52.7, "status": "OK" }
```

Поле status может принимать значения: "OK", "CRITICAL_TEMP", "CRITICAL_CPU", "CRITICAL_DIST".

3.3. Клиент (Windows, C++)

Назначение: подключение к серверу RPi, отправка команды, получение и отображение данных.

Используемые библиотеки: Winsock2 (Windows Sockets). Требуется линковка ws2_32.lib.

Функции:

- `parseJson()` – извлекает поля из JSON-строки (примитивный парсер на основе поиска подстрок).
- `receiveLine()` – читает из сокета до символа `\n`.
- `runLive()` – принимает поток JSON-строк, выводит их в консоль с цветовой индикацией статуса.
- `runLogs()` – принимает и выводит строки журнала событий.

Режимы работы:

1. LIVE – цветной вывод каждые 2 секунды: время, иконка состояния ([OK], [TEMP], [CPU], [DIST]), температура, загрузка CPU, память, диск, расстояние.
2. LOGS – вывод последних критических событий, сохранённых на сервере.

Завершение: по нажатию `Ctrl+C` (обработчик сигнала `SIGINT`) клиент корректно закрывает сокет и выходит.

4. Требования к оборудованию и программному обеспечению

4.1. Сервер (Raspberry Pi)

- ОС: Raspberry Pi OS (или любой дистрибутив Linux с поддержкой `/sys/class/thermal` и `/proc/stat`).
- Компилятор: `g++` с поддержкой C++11 и выше.
- Библиотеки: стандартная библиотека C++, `pthread` (используется неявно), `libstdc++`, `libc`.
- Порты: свободный USB-порт для подключения Arduino, доступ к сети (Ethernet/WiFi), открытый порт 5000 (TCP).
- Права доступа: чтение `/sys/class/thermal/thermal_zone0/temp`, `/proc/stat`, `/proc/meminfo`, а также чтение/запись `/dev/ttyUSB0` (пользователь должен быть в группе `dialout` или использовать `sudo`).

4.2. Arduino

- Плата: Arduino Uno / Nano / Mega (совместимая).
- Датчик: HC-SR04.
- Среда разработки: Arduino IDE 1.8+.
- Напряжение питания: 5 В (датчик подключается к 5 В и GND, сигнальные линии – к цифровым пинам 9 и 8).

4.3. Клиент (Windows)

- ОС: Windows 7/10/11 (32 или 64 бита).
- Среда выполнения: любой компилятор C++ с поддержкой Winsock2 (Microsoft Visual Studio, MinGW-w64, Clang).
- Сеть: доступ к IP-адресу сервера RPi по порту 5000.

5. Установка и настройка

5.1. Подготовка Arduino

1. Подключите HC-SR04 к Arduino:

- VCC → 5V
- GND → GND
- TRIG → цифровой пин 9
- ECHO → цифровой пин 8

2. Загрузите скетч (файл arduino.ino) через Arduino IDE.

3. Подключите Arduino к Raspberry Pi через USB-кабель.

4. Убедитесь, что устройство видно как /dev/ttyUSB0 (команда `ls -l /dev/ttyUSB*`).

Если номер другой – измените SERIAL_PORT в коде сервера.

5.2. Сборка и запуск сервера на Raspberry Pi

Установка компилятора (если не установлен)

```
sudo apt update
```

```
sudo apt install g++ make
```

Копирование исходного кода (например, pi_monitor_server.cpp)

```
g++ -o pi_monitor pi_monitor_server.cpp -std=c++11 -pthread
```

Запуск (лучше с правами root или добавить пользователя в группу dialout)

```
sudo ./pi_monitor
```

Сервер создаст директорию ~/pi_monitor/ и начнёт журналирование. Для автозапуска можно использовать systemd или cron.

5.3. Сборка и запуск клиента на Windows

- В Visual Studio: создайте консольный проект, добавьте файл client.cpp, укажите компоновку с ws2_32.lib.

- В командной строке (MinGW):

```
g++ -o pi_client client.cpp -lws2_32 -std=c++11
```

- Запуск:

```
pi_client.exe <IP_адрес_сервера>
```

Если IP не указан, используется 127.0.0.1 (локальный запуск для отладки). Укажите реальный IP вашего Raspberry Pi, например 192.168.1.100.

6. Протокол обмена данными

6.1. Последовательный интерфейс (Arduino ↔ RPi)

- Скорость: 9600 бод

- Строковый протокол: каждая строка заканчивается символом перевода строки (\n или \r\n).

- Формат сообщения: DIST:<число>

- Пример: `DIST:12.4`

- Некорректные или пустые строки игнорируются

6.2. TCP-протокол (RPi ↔ клиент)

- Порт: 5000, протокол TCP.

- Команды клиента:

- LIVE – переключить сервер в режим потоковой передачи.

- LOGS – запросить журнал событий.
- Ответы сервера:
 - В режиме LIVE: непрерывный поток JSON-строк, разделённых символом \n, каждые 2 секунды.
 - В режиме LOGS: сначала строка Reading critical events...\n, затем до 50 строк журнала, в конце --- End of Events Log ---\n.
 - При неизвестной команде: Unknown command. Use LIVE or LOGS.\n.

7. Функциональность и сценарии использования

7.1. Мониторинг в реальном времени (LIVE)

Пользователь выбирает режим 1. Клиент отправляет команду LIVE и начинает отображать обновляющиеся метрики. Экран очищается и каждые 2 секунды выводится строка с текущими значениями. Цветовое кодирование:

- Зелёный – всё в норме.
- Красный – перегрев (температура).
- Жёлтый – высокая загрузка CPU.
- Фиолетовый – объект слишком близко.

7.2. Просмотр критических событий (LOGS)

Пользователь выбирает режим 2. Клиент запрашивает журнал и выводит последние записи, например:

[EVENT] 14:23:10 OVERHEAT! Temp:71.2C (TEMP:71.2 CPU:45.3% DIST:35.0cm)

[EVENT] 14:25:03 OBJECT_CLOSE! Dist:15.7cm (TEMP:52.1 CPU:12.2% DIST:15.7cm)

Журнал хранится на сервере в файле critical_events.log и не очищается автоматически.

7.3. Система порогов и генерация событий

При каждом сборе метрик (каждые 5 секунд) сервер проверяет:

- Если температура > 70 °C – статус CRITICAL_TEMP.

- Иначе если CPU > 90 % – статус CRITICAL_CPU.
- Иначе если расстояние < 20 см – статус CRITICAL_DIST.
- Иначе – ОК.

При возникновении критического состояния в critical_events.log добавляется соответствующая запись. Даже если состояние сохраняется многократно, событие записывается при каждой проверке, что позволяет отследить длительность.

8. Примеры работы

Пример вывода клиента в режиме LIVE

LIVE MONITORING MODE

Press Ctrl+C to stop

[14:23:10] [OK] T:45.2C | CPU:12.5% | MEM:34.1% | DISK:28.3% | DIST:52.7cm

[14:23:12] [TEMP] T:71.1C | CPU:45.0% | MEM:34.2% | DISK:28.3% | DIST:53.0cm

[14:23:14] [DIST] T:45.5C | CPU:12.4% | MEM:34.3% | DISK:28.3% | DIST:15.3cm

Пример вывода клиента в режиме LOGS

CRITICAL EVENTS LOG

[EVENT] 14:23:10 OVERHEAT! Temp:71.2C (TEMP:71.2 CPU:45.3% DIST:35.0cm)

[EVENT] 14:25:03 OBJECT_CLOSE! Dist:15.7cm (TEMP:52.1 CPU:12.2% DIST:15.7cm)

9. Известные ограничения и возможные улучшения

- Производительность парсинга JSON – клиент использует примитивный поиск подстрок, что может ошибаться при наличии вложенных кавычек или изменении порядка полей. Рекомендуется заменить на полноценную JSON-библиотеку (например, nlohmann/json).

- Безопасность – соединение не зашифровано и не требует аутентификации. Для использования в открытых сетях необходимо добавить TLS или VPN.
- Обработка разрыва последовательного порта – при отключении Arduino сервер продолжает работу, но расстояние остаётся последним полученным значением. Можно добавить таймаут и сброс значения.
- Файл журнала критических событий – может неограниченно расти. Желательно добавить ротацию логов по размеру или времени.
- Многопоточность – использует `detach()` для потоков. При завершении программы ресурсы не освобождаются корректно. Для промышленного использования стоит добавить механизм `graceful shutdown`.

10. Заключение

Разработанная система позволяет в реальном времени контролировать состояние Raspberry Pi и получать оповещения о критических параметрах. Простота протокола и модульность компонентов дают возможность легко расширять функциональность (добавлять новые датчики, отправлять уведомления по электронной почте и т.д.). Документация охватывает все этапы развёртывания и использования.